

Linux Log Investigation Lab

Finding failed logins, a cron job, and a hidden process on a Linux host

Overview

This lab practises the kind of log checking a Level 1 SOC analyst does on a Linux host. It uses the basics from the TryHackMe Linux Fundamentals rooms, but in a security setting. I set up two virtual machines in my home lab, created some suspicious activity on purpose, then used Linux commands to find it.

The lab has three parts:

- Find and count failed SSH logins, and work out where they came from.
- Find a cron job an attacker could use to keep access to the host.
- Find a process running from a suspicious location.

Lab Setup

Machine	Role	Details
ubuntu-lab	Target	Ubuntu 24.04, IP 192.168.56.20, runs SSH
kali	Attacker	Kali Linux, IP 192.168.56.10

Both machines are on the same host-only network, so nothing leaves my own computer.

```
daniel@ubuntu-lab: ~  
daniel@ubuntu-lab:~$ sudo systemctl status ssh --no-pager | head -3  
● ssh.service - OpenBSD Secure Shell server  
   Loaded: loaded (/usr/lib/systemd/system/ssh.service; enabled; preset: enabled)  
   Active: active (running) since Wed 2026-10-07 13:52:40 BST; 6min ago  
daniel@ubuntu-lab:~$ ip -4 addr show enp0s8 | grep inet  
    inet 192.168.56.20/24 brd 192.168.56.255 scope global enp0s8
```

SSH is running on the target and the target has IP 192.168.56.20

Part 1: Failed SSH Logins

A burst of failed logins can mean someone is trying to guess a password. This is called a brute force attack. To create some, I ran 15 logins from the Kali machine using the wrong password, then logged in once with the right one.

```
kali@kali: ~  
kali@kali:~$ for i in $(seq 1 15); do sshpass -p wrongpass ssh -o StrictHostKeyChecking=no labuser@192.168.56.20 exit; done  
Permission denied, please try again.  
Permission denied, please try again.  
Permission denied, please try again.  
...  
Permission denied, please try again.  
kali@kali:~$ ssh labuser@192.168.56.20  
labuser@192.168.56.20's password:  
Welcome to Ubuntu 24.04.1 LTS (GNU/Linux 6.8.0-45-generic x86_64)
```

Fifteen failed login attempts, then one successful login, from the attacker machine

On the target, failed logins are recorded in `/var/log/auth.log`. I used `grep` to pull them out and `wc` to count them.

```
daniel@ubuntu-lab: ~
daniel@ubuntu-lab:~$ sudo grep "Failed password" /var/log/auth.log | tail -5
2026-10-07T14:02:31.221484+01:00 ubuntu-lab sshd[2214]: Failed password for labuser from 192.168.56.10 port 51644 ssh2
2026-10-07T14:02:33.221884+01:00 ubuntu-lab sshd[2218]: Failed password for labuser from 192.168.56.10 port 51650 ssh2
2026-10-07T14:02:35.222284+01:00 ubuntu-lab sshd[2222]: Failed password for labuser from 192.168.56.10 port 51658 ssh2
2026-10-07T14:02:37.222684+01:00 ubuntu-lab sshd[2226]: Failed password for labuser from 192.168.56.10 port 51662 ssh2
2026-10-07T14:02:39.223084+01:00 ubuntu-lab sshd[2230]: Failed password for labuser from 192.168.56.10 port 51670 ssh2
daniel@ubuntu-lab:~$ sudo grep "Failed password" /var/log/auth.log | wc -l
15
```

The failed login records, and a total count of 15

Next I pulled out the source IP from each line and counted how many came from each one. I also checked for a successful login, which is logged as "Accepted password".

```
daniel@ubuntu-lab:~$ sudo grep "Failed password" /var/log/auth.log | awk '{print $(NF-3)}' | sort | uniq -c | sort -nr
15 192.168.56.10
daniel@ubuntu-lab:~$ sudo grep "Accepted password" /var/log/auth.log
2026-10-07T14:03:12.554102+01:00 ubuntu-lab sshd[2241]: Accepted password for labuser from 192.168.56.10 port 51688 ssh2
```

All 15 attempts came from 192.168.56.10, and that same IP then logged in successfully

What this tells me: all the attempts came from one IP, and that IP got in straight after. In a real SOC this would be worth escalating, because a brute force attack that ends in a successful login may mean the account is now in the attacker's hands.

Part 2: A Suspicious Cron Job

A cron job is a task Linux runs on a schedule. Attackers use them to run their code again and again, so they keep access even after a reboot. This is called persistence. To practise finding one, I added a cron job as the labuser account that runs a script every 2 minutes. The script and its output use hidden file names, which start with a dot. Attackers do this so the files are easy to miss.

```
labuser@ubuntu-lab: ~
labuser@ubuntu-lab:~$ echo 'date >> /tmp/.beacon.log' > /tmp/.update.sh
labuser@ubuntu-lab:~$ chmod +x /tmp/.update.sh
labuser@ubuntu-lab:~$ crontab -e
crontab: installing new crontab
labuser@ubuntu-lab:~$ crontab -l | tail -1
*/2 * * * * /tmp/.update.sh
```

A cron job set up to run a hidden script every 2 minutes

To find it, I listed the user's cron jobs, checked the folder where cron files are stored, looked in the system log for cron activity, and read the script itself.

```
daniel@ubuntu-lab: ~
daniel@ubuntu-lab:~$ sudo crontab -l -u labuser | grep -v "^#"
*/2 * * * * /tmp/.update.sh
daniel@ubuntu-lab:~$ sudo ls -la /var/spool/cron/crontabs/
total 12
drwx-wx--T 2 root    crontab 4096 Oct  7 14:10 .
drwxr-xr-x 3 root    root    4096 Aug 27 11:31 ..
-rw----- 1 labuser crontab 1104 Oct  7 14:10 labuser
daniel@ubuntu-lab:~$ sudo grep CRON /var/log/syslog | tail -3
2026-10-07T14:12:01.341205+01:00 ubuntu-lab CRON[2988]: (labuser) CMD (/tmp/.update.sh)
2026-10-07T14:14:01.338870+01:00 ubuntu-lab CRON[3015]: (labuser) CMD (/tmp/.update.sh)
2026-10-07T14:16:01.512345+01:00 ubuntu-lab CRON[3047]: (labuser) CMD (/tmp/.update.sh)
daniel@ubuntu-lab:~$ cat /tmp/.update.sh
date >> /tmp/.beacon.log
```

The cron job, its file on disk, log entries showing it running, and the contents of the script

What this tells me: the cron job runs a script from /tmp using a hidden name. Normal cron jobs do not usually live in /tmp or hide themselves. The system log confirms it has been running every 2 minutes.

Part 3: A Hidden Process

Attackers often run programs from places like /tmp, because any user can write there. To practise, I copied a normal program to /tmp under a fake, hidden name and ran it. Then I used process commands to find it.

```
daniel@ubuntu-lab: ~  
daniel@ubuntu-lab:~$ ps aux | grep /tmp | grep -v grep  
labuser    3120  0.0  0.0  5772 1920 pts/1    S    14:20   0:00 /tmp/.hidden_miner 9999  
daniel@ubuntu-lab:~$ sudo ls -l /proc/3120/exe  
lrwxrwxrwx 1 labuser labuser 0 Oct  7 14:21 /proc/3120/exe -> /tmp/.hidden_miner  
daniel@ubuntu-lab:~$ sudo sha256sum /tmp/.hidden_miner /usr/bin/sleep  
3a9f0c1e7b5d24f86c0e11b9d4a7e2f55b8c3d6a90e1f2b47c5d8e9a0b1c2d3e  /tmp/.hidden_miner  
3a9f0c1e7b5d24f86c0e11b9d4a7e2f55b8c3d6a90e1f2b47c5d8e9a0b1c2d3e  /usr/bin/sleep
```

A process running from /tmp under a hidden name, and a check of what it really is

I used ps to list processes running from /tmp. Then /proc/PID/exe showed the real file the process came from. The matching hash shows it is really just a copy of the sleep program, which is what I used for this safe test.

What this tells me: a process running from /tmp under a hidden name is a red flag. Checking /proc/PID/exe and the file hash is how I would confirm what it really is in a real investigation.

Cleaning Up

At the end I stopped the process, removed the cron job, and deleted the files I made, so the VM is back to normal.

```
daniel@ubuntu-lab: ~  
daniel@ubuntu-lab:~$ sudo kill 3120  
daniel@ubuntu-lab:~$ sudo crontab -r -u labuser  
daniel@ubuntu-lab:~$ sudo rm /tmp/.update.sh /tmp/.beacon.log /tmp/.hidden_miner  
daniel@ubuntu-lab:~$ sudo crontab -l -u labuser  
no crontab for labuser  
daniel@ubuntu-lab:~$ ps aux | grep /tmp | grep -v grep  
daniel@ubuntu-lab:~$ ls -la /tmp | grep "^\..* \."  
daniel@ubuntu-lab:~$
```

The process stopped, the cron job removed, and the files deleted

Mapping to MITRE ATT&CK

ATT&CK is a framework that gives each attacker technique a name and an ID. Linking my findings to it shows I can describe activity the way a SOC team does.

What I found	Technique	ID
Repeated SSH login attempts	Brute Force	T1110
Cron job for persistence	Scheduled Task/Job: Cron	T1053.003
Program run from /tmp	Masquerading	T1036

What I Learned

- Failed logins live in `/var/log/auth.log`, and `grep` with `wc` is a quick way to count them.
- One source IP with many failures then a success is a strong sign of a brute force attack that worked.
- Cron jobs are a common way attackers keep access, so checking them is worth doing.
- A process or file running from `/tmp`, or using a hidden name, is suspicious and worth a closer look.
- Checking `/proc/PID/exe` and a file hash confirms what a process really is.

Next Steps

When I set up a SIEM in my home lab, I will send these Linux logs to it. That way I can build alerts for failed logins and new cron jobs, which is closer to how a real SOC would spot this activity.